Modeling and Calibrating: LLMs Figure out their Decoding Process

Matthew Khoriaty

Reliable ML with Unreliable Black Boxes

June 4, 2026

The setup: a model has an incentive to know its own decoder

- An autoregressive LM is trained to *predict* text. When it *generates*, a **decoder** turns its logits into tokens — temperature, top-k, a banned-token list.
- To keep predicting its own output well, it has an incentive to **model that decoder** from the text it has already produced.

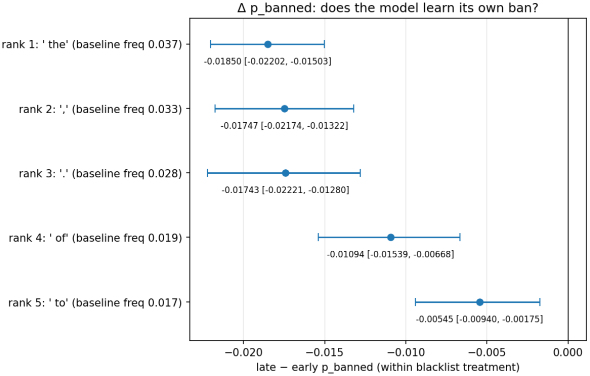
Three questions

1. Does the model model its decoder? **yes**
2. Can we use that to *predict* text better? **yes / no**
3. Can we use that to *generate* better? **not yet**

Everything below uses only the model's own logits — training-free.

Q1: does the model model its decoder? **Yes.**

Constrain the decoder while it generates, then ask whether the model — reading back its own constrained text — has *inferred* the constraint.

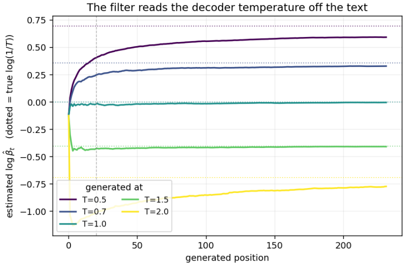


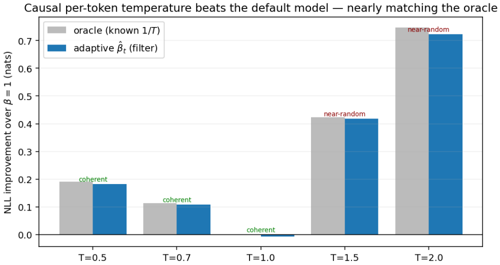
- **Blacklist.** Ban a token. The model lowers its predicted probability on that token over the sequence ($\Delta p_{\text{late–early}} = -0.019$ for the top token), and below an unbanned control (-0.016, 95% CI $[-0.019, -0.013]$). The early-position contrast is ≈ 0 (sanity).
- **Top-k.** Truncate sampling to the top k. The model piles extra mass right at the rank-k boundary: treat.$-$ctrl $= +0.475$ (k=3), $+0.170$ (k=5), CIs clean of 0.

The model tracks discrete decoder constraints. Next: a **continuous** *one — temperature.*

Q2: can we use that to *predict* better? (1/2: known temperature)

A streaming Bayesian filter reads an inverse-temperature $\hat{\beta}_t = 1/\hat{T}_t$ off the prefix, *one token at a time*, causally. Test it on Qwen-2.5-3B self-text generated at a *known* temperature (64 sequences per T).





- **Recovery (left):** $\hat{\beta}_t$ settles on the true $\log(1/T)$ from the prefix alone (bias -0.030 at $T{=}0.7$).
- **Prediction (right):** using $\hat{\beta}_t$ as a per-token temperature beats the default $\beta{=}1$ model by 0.108 nats at $T{=}0.7$ (up to 0.723 at $T{=}2$), within $+0.025$ nats of the oracle that *knows* $1/T$. *(Honesty: the big $T \geq 1.5$ gains are on near-random text; coherent wins are $T{=}0.5, 0.7$.)*

Q2: can we use that to *predict* better? (2/2: real text)

That was text with a temperature *put* in it. Does *natural* text carry one? Fit the best global temperature per text type; measure held-out NLL gain over β=1, across 9 types from in-distribution to far-OOD.

text type	optimal T	ΔNLL vs β=1 (nats)	β=1 NLL	$H_{\beta=1}$
shuffled WikiText	1.08	$+0.0283^{**}$	7.20	6.48
Recamán (A005132)	0.91	$+0.0032^{**}$	0.08	0.14
ABC music	1.03	$+0.0019^{**}$	2.48	2.34
WritingPrompts	1.04	+0.0011	2.87	2.74
Telugu	1.03	$+0.0010^{**}$	1.12	1.06
Esperanto	1.03	+0.0007	2.72	2.63
Yoruba	1.00	+0.0001	3.06	3.11
WikiText	1.00	-0.0000	1.96	1.93
Gutenberg (archaic EN)	1.01	-0.0001	1.87	1.85

- **Qwen is temperature-robust everywhere:** optimal $T \in [0.91, 1.08]$; the best temperature buys $\leq +0.0032$ nats on any real type ($+0.0283$ on shuffled gibberish). Recamán (an integer sequence) is the lone exception — uniquely *under*confident. "Harder $\neq$ mis-temperatured": Yoruba is hardest, yet $T \approx 1$.
- $\Rightarrow$ **yes** on temperatured text, **no** in the wild — there is essentially nothing to recalibrate.

Q3: can we use that to *generate* better? **Not yet.**

- The tempting move: feed $\hat{\beta}_t$ *back* into the sampler to "correct" the temperature as it generates.
- **It self-neutralizes.** Correcting toward $\hat{\beta}$ drives the effective decoding temperature back to $\beta_{\mathrm{dec}} \to 1$ — the very signal we read off is quietly cancelled. Confirmed across exact-grid filters, so it is not an approximation artifact.

So the read-out is real and usable for **prediction**, *but turning it into better* **generation** *is still open — the honest "in progress."*

Takeaways

- **The model carries a real, training-free signal about its own decoder**, readable from its logits alone — it tracks a blacklist, a top-k cutoff, and a continuous temperature.
- That signal lets us **predict** known-temperature text materially better, nearly matching an oracle that knows the temperature.
- Two honest limits: modern LLMs are so **temperature-robust on real text** (even far out of distribution) that there is little to fix in the wild; and using the signal to **generate** better is unresolved — closed-loop correction cancels itself.